SellingDocs 기획 리뷰

리뷰 일자: 2026-10-12 / 대상 경로: D:₩WorkSpace₩platform₩구현해야 할 프로젝트₩SellingDocs (코드 없음, 기획 문서만 존재)

1. 프로젝트 개요

- 한 줄 설명: 원고를 입력하거나 AI로 생성한 뒤, 한 번의 클릭으로 인쇄용 PDF(WeasyPrint)와 리플로우 ePub(epub-gen-memory)을 동시에 빌드하는 1인 디지털 출판 SaaS MVP (PRD.md:11-12).
- 기획 의도: API/백엔드 공수를 최소화하고 Next.js Route Handlers + Supabase(Auth, RLS)만으로 솔로 개발자가 "바이브 코딩"으로 빠르게 출시 (PRD.md:13).
- 대상 사용자: 원고를 가진 1인 작가, 전자책을 판매하려는 개인 크리에이터. 문서에 명시적인 페르소나 정의는 없다.

영역	제안된 기술	근거
프론트	Next.js 14+ (App Router), Tailwind, shadcn/ui	PRD.md:21-23
백엔드	Next.js Route Handlers (app/api/generate, export/pdf, export/epub)	PRD.md:125-131
DB/인증	Supabase PostgreSQL + Auth(매직링크, Google) + RLS	PRD.md:22,47,72-74
인프라	Vercel	PRD.md:24
출판 엔진	WeasyPrint(PDF), epub-gen-memory(ePub), QuickChart API(차트 이미지)	PRD.md:28-30
AI	OpenAI, Anthropic, Gemini, OpenRouter 4종 직접 연동 후 동적 스위칭	PRD.md:34-39

- 기획 완성도: 약 35%.
  - 근거: 기술 스택, DDL 2개 테이블과 RLS, 디렉토리 구조, 구조화 JSON 응답 스키마는 구체적이다. 그러나 add.md와 Claude.md는 0바이트 빈 파일이고, PRD.md도 9KB 분량으로 사용자 플로우, 화면 명세, API 요청/응답, 크레딧 단가, 결제, 에러 처리, 비기능 수치(응답 시간, 동시성)가 전혀 없다. 무엇보다 핵심 엔진(WeasyPrint)이 제안된 런타임(Next.js + Vercel)에서 동작하지 않는 구조적 모순이 있어 설계 확정 전 단계로 본다.

2. 기획 문서 분석

파일	크기	역할	상태
PRD.md	9,201B, 159행	개요, 스택, 기능/비기능 요구사항, DDL, 디렉토리 구조, Claude Code 실행 프롬프트	유일한 실질 문서. 첫 3행과 마지막 블록은 AI 대화 응답 문구가 그대로 남아 있음(PRD.md:1-3,149-157)
Claude.md	0B	(의도상) Claude Code 개발 가이드	비어 있음
add.md	0B	(의도상) 추가 요구사항	비어 있음

요구사항 구조 요약:

- 기능 요구사항 4개 묶음: 인증/크레딧(3.1), AI 책 기획 빌더(3.2, 입력 6필드 + 모델 4종 택1 + 챕터 JSON 스키마), 편집기/프리뷰(3.3, 여백/패딩 슬라이더 + QuickChart 차트 삽입), 내보내기(3.4, PDF/ePub).
- 비기능 요구사항 2개: 전 테이블 RLS(4.1), GEO용 JSON-LD 메타(4.2).
- 데이터: profiles(credits 10 기본), books(chapters_json jsonb) 2개 테이블.

sellingstudio 와의 관계 판단: 별개 제품 (같은 "Selling" 브랜드의 자매 기획)

- sellingstudio는 쿠팡/스마트스토어 셀러용 "AI 상세페이지 빌더"(카피 + fal.ai 이미지 생성, docs/reviews/sellingstudio.md:7-8)이고, SellingDocs는 "전자책 PDF/ePub 출판" 도구다. 입력, 산출물, 대상 사용자, DB 스키마(selling_pages/image_jobs vs books)가 모두 다르므로 원본 기획도 확장 기획도 아니다.
- 공통점은 아키텍처 골격뿐이다: Next.js + Supabase Auth/RLS + 크레딧 차감 + 다중 LLM + 구조화 JSON 응답. 따라서 "코드 재사용 대상"으로서의 관계는 강하다(4장).
- 계보상 확인된 관계는 두 가지다.
  1. FortuneDocs/PRD.MD:3이 "셀링독스의 견고한 아키텍처를 계승하고, 파이썬 없이"라고 명시한다. 즉 SellingDocs는 FortuneDocs의 원형 템플릿이며, FortuneDocs는 SellingDocs의 WeasyPrint를 Puppeteer로 교체했다(FortuneDocs/PRD.MD:84). 이 사실은 아래 문제 #1을 작성자도 인지했음을 보여준다.
  2. 기능 도메인이 가장 많이 겹치는 것은 sellingstudio가 아니라 Flora-v2다. Flora-v2는 1인 출판 AI SaaS로 PDF/ePub 내보내기(Flora-v2/api/routers/export.py:23-53), WeasyPrint 외부 서버 호출(export.py:27-35), 여백 슬라이더(hooks/use-doc-styles.ts, Auto-Fit), 원고 업로드 파싱, Stripe 결제를 이미 기획/일부 구현했다. SellingDocs의 기능 범위는 Flora-v2 PRD의 부분집합이다.

3. 기획상의 문제점

#	심각도	분류	위치	내용
1	높음	기술 리스크	PRD.md:13,24,28,65,130	WeasyPrint는 Python + Pango/Cairo 네이티브 라이브러리가 필요하다. "Next.js Route Handlers만으로" + Vercel 배포라는 전제와 양립 불가. export/pdf/route.ts(TypeScript)에서 WeasyPrint를 호출할 방법이 기술되지 않음
2	높음	보안/누락	PRD.md:48, PRD.md:86-91,111-112	크레딧 "차감"의 주체와 방식이 없다. 단가표, 차감 시점(요청 전/성공 후), 실패 환불, 원자적 차감 RPC, credit_logs 장부가 모두 누락. sellingstudio에서 실제로 발생한 "생성 라우트가 크레딧을 확인하지 않음 / 비원자 차감"(sellingstudio.md:58-59) 결함이 그대로 재현될 설계
3	높음	누락	Claude.md, add.md	두 문서가 0바이트. PRD가 지시하는 바이브 코딩 흐름(PRD.md:157)에서 에이전트 규칙(코딩 컨벤션, Supabase 클라이언트 패턴, 금지사항)이 없어 생성 코드 품질이 통제되지 않음
4	높음	기술 리스크	PRD.md:53-54, PRD.md:128	책 전체를 단일 JSON 응답({"title","chapters":[...]})으로 받는 구조. 출력 토큰 한도와 Vercel 함수 실행 시간 제한에 걸려 수 챕터 이상이면 잘림/타임아웃/JSON 파싱 실패가 발생. 목차 생성 -> 챕터별 순차 생성(스트리밍 또는 작업 큐) 설계가 필요
5	중간	모순	PRD.md:78 vs PRD.md:115-116	"오픈 서가(Public Bookshelf)"를 GEO 대상으로 명시하지만 books에는 공개 여부 컬럼이 없고 RLS는 소유자 전용 FOR ALL뿐이라 공개 페이지가 구조적으로 불가능
6	중간	모순	PRD.md:60 vs PRD.md:29,66	여백/패딩 슬라이더는 고정 판형 PDF에만 의미가 있다. ePub은 리플로우 포맷이라 리더기가 여백을 결정한다. 같은 편집 설정이 두 포맷에 어떻게 매핑되는지 정의 없음
7	중간	모호성	PRD.md:59-61, PRD.md:102	"위지윅 에디터"가 실제로는 슬라이더 2개만 정의됨. 본문 편집기(Tiptap 등) 유무, 저장 포맷(HTML인지 Markdown인지 chapters_json 주석부터 "HTML/마크다운"으로 미정), 자동 저장, 버전 관리가 없음
8	중간	보안	PRD.md:61,65,102	AI 생성 HTML과 사용자 <img>를 그대로 저장/렌더/PDF 컴파일. XSS 위생 처리(sanitize) 규칙이 없고, HTML->PDF 엔진이 외부 URL을 가져오므로 SSRF(내부망 이미지 URL) 방어 정책 필요
9	중간	누락	PRD.md:12, PRD.md:99	"원고 업로드"와 "표지 이미지"를 언급하지만 업로드 포맷, Supabase Storage 버킷, 용량 제한, 파싱 규칙이 없음

#	심각도	분류	위치	내용
10	중간	범위 과다	PRD.md:34-39	MVP에 LLM 공급자 4곳 직접 연동(키 4개, SDK 4개, 응답 포맷 4종). 모델 선택을 사용자에게 노출하면 크레딧 단가도 모델별로 달라져야 하는데 정의 없음. OpenRouter 하나로 4종 모두 호출 가능
11	중간	기술 리스크	PRD.md:28-30,65-66	한글 폰트 임베딩(Noto Serif/Sans KR) 언급 없음. 서버리스 환경의 PDF 엔진과 ePub 모두 CJK 폰트를 명시적으로 번들해야 하며 미지정 시 두부 문자 출력
12	낮음	모순	PRD.md:53 vs PRD.md:98	입력 모델에 저자명이 없는데 books.author_name은 NOT NULL
13	낮음	누락	PRD.md:86-104	profiles 자동 생성 트리거, books.updated_at/status, 내보내기 이력 테이블, 결제 테이블 없음. 결제 유도 모달(PRD.md:48)은 있으나 PG(Stripe/Toss) 미정
14	낮음	보안	PRD.md:30,61	QuickChart 공개 API로 사용자 데이터를 외부 전송. "데이터 원천 격리"(PRD.md:74) 원칙과 상충. 서버에서 차트 이미지를 받아 Storage에 저장 후 임베딩해야 ePub 오프라인 표시도 보장
15	낮음	기술 리스크	PRD.md:21	Next.js 14+ 전제이나 워크스페이스 표준은 Next 16(basic-master-module/package.json next 16.2.4). 16에서는 middleware.ts가 proxy.ts로 바뀌는 등 PRD 가이드가 뒤처짐

심각도 높음 상세

#1 WeasyPrint와 런타임 불일치. PRD는 "Next.js Route Handlers와 Supabase 내장 기능 조합"으로 백엔드를 대체한다고 하면서 PDF 엔진으로 Python 라이브러리 WeasyPrint를 지정했다. Vercel Node 함수에는 Python이 없고, Vercel Python 함수도 Pango 같은 시스템 라이브러리가 없어 WeasyPrint가 설치되지 않는다. 선택지는 (a) WeasyPrint를 Docker 컨테이너(Cloud Run/Railway/Fly)로 분리해 POST /pdf {html} 마이크로서비스로 두기(Flora-v2가 WEASYPRINT_SERVER_URL로 택한 방식), (b) FortuneDocs처럼 Puppeteer + @sparticuz/chromium으로 전환, (c) 브라우저 측 Paged.js 프리뷰 + 서버 Chromium 출력이다. CSS Paged Media 품질(@page :left/:right, 러닝 헤더)을 원한다면 (a)가 맞고, 워크스페이스에 이미 WeasyPrint - CSS Paged Media 명세.md가 있으므로 (a)를 권장한다.

#2 크레딧 과금 경계 부재. profiles.credits에 CHECK (credits >= 0)만 있고 차감 경로가 없다. SELECT 정책만 있어 클라이언트 UPDATE는 막히므로 서버가 service_role로 갱신해야 하는데, 이 경우 read-then-write로 구현하기 쉽다. deduct_credits(cost, reason) SECURITY DEFINER 함수 하나로 UPDATE ... WHERE credits >= cost RETURNING + credit_logs insert를 원자 처리하고, generate/export 라우트 내부에서 "차감 -> 실행 -> 실패 시 환불"을 수행하도록 PRD에 명문화해야 한다.

#3 빈 Claude.md/add.md. 바이브 코딩을 전제로 한 프로젝트에서 에이전트 규칙 문서가 비어 있으면 매 세션 다른 구조가 생성된다. 최소한 Supabase 클라이언트 3종 패턴, Route Handler 순서(인증 -> 크레딧 -> 처리), zod 검증, 금지사항(클라이언트 직접 insert 금지 등)을 채워야 한다.

#4 단일 JSON 책 생성. 한 번의 LLM 호출로 책 전체를 받는 설계는 데모 수준에서만 동작한다. 목차 JSON 생성(1회) 후 챕터별 생성 요청을 순차/병렬로 보내고 진행률을 표시하는 2단계 파이프라인이 필요하며, 각 단계의 크레딧 단가도 분리해야 한다.

4. 효율적 구현 전략

권장 아키텍처

- Next.js 16 App Router(Vercel) + Supabase(Auth, Postgres, Storage, RLS) + **PDF 렌더 마이크로서비스 1개**(WeasyPrint Docker, Cloud Run 등, 공유 시크릿 헤더로만 호출).
- ePub은 Node의 epub-gen-memory로 Route Handler에서 처리(Vercel에서 동작). 이미지(표지, 차트)는 Storage URL로 받아 ePub 안에 바이너리 포함.

- AI는 OpenRouter 단일 게이트웨이. 사용자 노출 모드는 "빠름/고품질" 2종으로 줄이고 내부에서 모델 매핑.
- 생성 파이프라인: POST /api/books/{id}/outline(목차) -> POST /api/books/{id}/chapters/{n}(챕터별) -> 각 라우트 안에서 deduct_credits RPC 호출.
- 데이터: books에 status, updated_at, is_public, slug, layout_settings jsonb(여백/판형) 추가, chapters를 별도 테이블로 분리(챕터 단위 편집/재생성), credit_logs, exports(파일 경로, 포맷) 추가.

재사용할 기존 프로젝트/모듈

출처	재사용 대상	용도
basic-master-module	Next 16.2.4 + React 19 + Tailwind v4 + ESLint 골격, AGENTS.md 규칙	새 프로젝트 시작점. 이것을 복제 후 shadcn 초기화
sellingstudio/lib/supabase/{client,server,admin}.ts, proxy.ts, lib/proxy.ts	Next 16 대응 Supabase SSR 클라이언트 3종 + 세션 갱신/라우트 보호	그대로 복사
sellingstudio/app/api/auth/callback/route.ts	Google OAuth 콜백 + profiles 보장	복사하되 next 화이트리스트 보완(sellingstudio.md:64)
sellingstudio/lib/llm/openrouter.ts, prompts.ts	OpenRouter 호출 래퍼, 구조화 JSON 프롬프트 패턴	복사 후 타임아웃 추가
sellingstudio/supabase/migrations/001_initial_schema.sql	profiles + credit_logs + 가입 트리거 + RLS	구조 차용. 단 USING(true) UPDATE 정책(sellingstudio.md:61)은 제외
sellingstudio 리뷰 작업 1의 deduct_credits RPC	원자 차감/환불	sellingstudio에 먼저 구현 후 양쪽 공용
Flora-v2/api/routers/export.py:56-173	_build_html(워터마크, docStyles 반영 HTML 조립), _generate_epub	PDF 서비스용 HTML 템플릿과 ePub 구조 참조
Flora-v2 Auto-Fit 프리셋(PRD.md 8장, lib/auto-fit/auto-fit-presets.ts), hooks/use-doc-styles.ts	판형별 여백/폰트 프리셋, 슬라이더 동기화	여백 슬라이더 기능을 판형 프리셋으로 대체
WeasyPrint - CSS Paged Media 명세.md	@page, 좌우 페이지, 러닝 헤더 CSS	PDF 템플릿 CSS 기준

MVP 범위 축소안

- 유지: 매직링크/Google 로그인, 책 생성(목차 -> 챕터 순차), 챕터 본문 편집(Tiptap, HTML 저장), 판형 프리셋 3종(A5/B5/신국판), PDF/ePub 내보내기, 크레딧 차감/로그, 대시보드 책 목록.
- 연기: QuickChart 차트(2차), 4종 모델 선택 UI(2종 모드로 축소), 오픈 서가/GEO JSON-LD(2차), 원고 파일 업로드(.md/.txt만 1차 허용, docx는 2차), 결제(1차는 수동 크레딧 충전, 2차 Stripe/Toss).
- 결정 필요: Flora-v2와 제품을 분리할지. Flora-v2가 같은 기능을 더 넓게 기획하고 있으므로 SellingDocs를 "Flora-v2의 경량 Next 단독판"으로 포지셔닝하거나 Flora-v2에 흡수하는 쪽이 중복 개발을 줄인다.

단계별 로드맵

1. 0단계(설계 확정, 2~3일): PDF 엔진 결정, PRD 보강(크레딧 단가, 사용자 플로우, 공개 서가 정책), Claude.md 작성, DDL v2.
2. 1단계(골격, 1주): basic-master-module 복제 -> sellingstudio Supabase/인증/proxy 이식 -> 마이그레이션(profiles, books, chapters, credit_logs, exports, deduct_credits) -> 대시보드.
3. 2단계(핵심, 1~2주): 목차/챕터 생성 라우트, Tiptap 편집기, 판형 프리셋, ePub 라우트, WeasyPrint Docker 서비스 + PDF 라우트, 한글 폰트 번들.
4. 3단계(확장): QuickChart(서버 저장 방식), 공개 서가 + JSON-LD, 결제, 원고 업로드 확장.

5. 구현 작업 목록 및 권장 모델

#	작업	우선순위	규모	권장 모델	이유
1	PDF 엔진 결정(WeasyPrint 컨테이너 vs Puppeteer) 및 전체 아키텍처/데이터 모델 v2 설계(chapters 분리, layout_settings, is_public, exports), PRD 보강	P0	M	Fable 5.1	기획안만 있는 프로젝트의 설계, 런타임 모순 해소
2	Claude.md 작성(스택 버전, Supabase 패턴, 라우트 순서 규칙, 금지사항), add.md 정리 또는 삭제	P0	S	Sonnet 5.5	1번 결과를 문서화하는 작업
3	basic-master-module 복제, shadcn 초기화, sellingstudio의 lib/supabase/*, proxy.ts, auth callback 이식(next 화이트리스트 포함)	P0	S	Opus 5.5	정의된 이식 작업, 인증 경로라 Sonnet보다 한 단계 위
4	마이그레이션 001: profiles(가입 트리거), books, chapters, credit_logs, exports + 전 테이블 RLS + 공개 서가용 SELECT 정책 + deduct_credits/refund_credits SECURITY DEFINER 함수	P0	M	Fable 5.1	과금/권한 로직, 실수 시 금전/데이터 피해
5	목차 생성 -> 챕터별 생성 Route Handler(OpenRouter, zod 출력 검증, 타임아웃, 크레딧 차감/실패 환불)	P0	M	Fable 5.1	생성 파이프라인 설계와 과금 경계를 함께 다룸
6	WeasyPrint Docker 서비스(POST /pdf, 시크릿 헤더, 외부 URL fetch 차단 url_fetcher, Noto 폰트 포함) + Next export/pdf 라우트	P0	M	Opus 5.5	명확한 서비스 구현 + API 연동
7	export/epub 라우트(epub-gen-memory, 표지/이미지 바이너리 포함, 한글 폰트 임베드), Storage 저장 및 exports 기록	P1	M	Opus 5.5	정의된 기능 구현
8	판형 프리셋 3종 + 여백 슬라이더(Flora-v2 Auto-Fit 값 차용), PDF용 Paged Media CSS 템플릿(좌우 페이지, 쪽번호, 러닝 헤더)	P1	M	Opus 5.5	명세 문서 기반 중간 규모 구현
9	챕터 편집기(Tiptap), 자동 저장(debounce), HTML sanitize(서버 측 DOMPurify 등)	P1	M	Opus 5.5	상태관리와 XSS 처리 포함
10	대시보드(책 목록/생성/삭제), 크레딧 잔액 표시와 부족 시 안내 모달, 랜딩 페이지	P1	M	Sonnet 5.5	반복적 UI 작업
11	QuickChart 차트 삽입: 서버에서 이미지 생성 -> Storage 저장 -> 본문 임베딩	P2	S	Opus 5.5	외부 API 연동
12	공개 서가 페이지 + Metadata API/JSON-LD(Book 스키마), sitemap	P2	S	Sonnet 5.5	보일러플레이트 성격
13	결제(Stripe 또는 Toss) 크레딧 충전 + webhook 서명 검증	P2	L	Fable 5.1	결제 로직
14	크레딧 RPC 동시성 테스트, 생성/내보내기 라우트 통합 테스트, .env.example + lib/env.ts zod 검증	P1	S	Opus 5.5	테스트 설계

6. 종합 평가

- 실현 가능성: 중간. 범위 자체는 솔로 개발로 3~4주 MVP가 가능한 크기이나, 현재 PRD 그대로는 PDF 엔진이 배포 환경에서 동작하지 않고 크레딧 과금 경계가 비어 있어 "문서대로 바이브 코딩" 시 출시 불가 상태가 된다. 설계 보강(작업 1, 4) 후에는 sellingstudio와 Flora-v2 자산 재사용으로 실현 가능성이 높아진다.
- 강점: 스택이 워크스페이스 표준(Next + Supabase)과 일치해 재사용 자산이 많다. 구조화 JSON 응답과 RLS를 처음부터 전제했다. 기능 범위가 Flora-v2보다 훨씬 작아 MVP로 적절하다.
- 약점: PDF 엔진 런타임 모순, 과금 설계 부재, 보조 문서 2개 공백과 화면/플로우 명세 부재.
- 모델별 비율(작업 14개 기준): Fable 5.1 29% (4건) / Opus 5.5 50% (7건) / Sonnet 5.5 21% (3건)

- 권장 진행 순서: 작업 1 -> 2 -> 3/4 -> 5/6 -> 7~10, 14 -> 11~13.
- 주 담당 모델: **Fable 5.1**. 작업 수로는 Opus 5.5가 가장 많지만, 이 프로젝트는 코드가 없는 기획 단계이고 성패가 "PDF 엔진을 어디서 돌릴지", "크레딧 차감을 어떤 트랜잭션 경계로 묶을지", "챕터 단위 생성 파이프라인과 데이터 모델"이라는 초기 설계 결정에 달려 있다. 이 결정을 Fable 5.1이 내리고 초기 골격(마이그레이션, RPC, 생성 라우트)까지 구현한 뒤, 정의된 내보내기/편집기/연동 작업은 Opus 5.5, UI와 문서는 Sonnet 5.5로 넘기는 것이 비용 대비 위험이 가장 낮다.